

Trabalho 1 — Concorrência e Sincronização: O Problema dos Filósofos Jantando

Grupo: [NOME 1 — matrícula] · [NOME 2 — matrícula] · [NOME 3 — matrícula] | **Disciplina:** Fundamentos de Programação Paralela e Distribuída (PUCRS) | **Código-fonte (Git):** https://github.com/Ilkalil/T1_FPPD

Soluções, mecanismos de sincronização e como são implementados em Go

Cinco (ou N) filósofos, um garfo entre cada par. O filósofo i usa os garfos i (esquerdo) e $(i+1) \bmod N$ (direito). Cada garfo é um `sync.Mutex` (Lock/Unlock: exclusão mútua, sem preempção e não reentrante); cada filósofo é uma goroutine e `sync.WaitGroup` aguarda o fim. Pensar/comer é `time.Sleep` (10 ms cada), igual em todas as versões, e as métricas usam `sync/atomic` (contador de filósofos comendo, com máximo via CAS) e medição por filósofo do tempo entre o fim de pensar e a posse dos dois garfos. Um *watchdog* declara deadlock se nenhum ciclo termina por 5 s e imprime quem segura e quem espera cada garfo (o detector nativo do runtime não dispara porque há timers ativos).

- **V0 — sequencial (baseline):** um filósofo por vez, sem goroutines nem garfos; o tempo esperado é $N \times 20 \times (10+10)$ ms.
- **V1 — deadlock:** todos pegam primeiro o garfo esquerdo e depois o direito, com pausa de 1 ms entre os dois (-gap, só nesta versão) para tornar o deadlock provável. Se todos seguram o esquerdo, ninguém avança.
- **V2 — sincronização excessiva:** um mutex global é adquirido antes dos garfos e só liberado depois de comer. Correta e sem deadlock, mas serializa a mesa inteira: só um filósofo come por vez, mesmo havendo garfos livres.
- **V3 — aprimorada: hierarquia de recursos:** cada filósofo pega sempre o garfo de menor índice primeiro. O último filósofo (N-1) inverte a ordem (garfo 0 antes do N-1), o que quebra a espera circular; vizinhos não conflitantes comem em paralelo.

V1: por que ocorre deadlock (as 4 condições de Coffman)

Exclusão mútua: um garfo (Mutex) só tem um dono. **Segura-e-espera:** cada filósofo segura o garfo i e espera o $i+1$. **Ausência de preempção:** o Mutex não pode ser tomado à força. **Espera circular:** $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$ (o filósofo i espera o garfo em posse do $i+1$). Com as quatro presentes o sistema trava: cada filósofo tem **1 garfo** (o esquerdo) e espera **1** (o direito, do vizinho); ver captura na página 2.

Como V2 e V3 evitam o deadlock; starvation

V2 nega *segura-e-espera/espera circular*: só quem tem o mutex global pede garfos, e como ninguém mais os segura eles sempre estão livres. **V3** nega a *espera circular*: ordenando os recursos, a cadeia de esperas é sempre crescente e não fecha um ciclo. **Starvation:** em V2 e V3 o `sync.Mutex` do Go entra em modo de inanição após 1 ms de espera (entrega direta em FIFO ao mais antigo), o que limita a espera de cada goroutine; observamos todos os filósofos concluindo os 20 ciclos em todas as execuções. Em teoria a V3 sozinha, com escalonamento arbitrário, não garante justiça; aqui ela vem do mutex.

Ambiente de execução

AWS EC2 **c8g.16xlarge** (spot, us-east-1b): AWS Graviton4, **64 vCPUs** (1 thread/core), Amazon Linux 2023 (kernel 6.18, aarch64), **Go 1.26.8 linux/arm64**. Compilação: `go build -o filosofos .` Execução: `NS="5 48 96" RUNS=5 ./run.sh` (V1: 10 execuções por N; individual: `./filosofos -v 3 -n 96 -csv`). 20 ciclos por filósofo, pensar = comer = 10 ms, 5 execuções por versão e N (o enunciado pede N=5; N=48 e 96 foram acrescentados para ver a escala). Como pensar/comer são *sleep*, o resultado não depende do número de cores (tentamos a c8g.48xlarge/24xlarge, sem capacidade spot).

Visão geral das versões

Versão	Sincronização	Deadlock?	Máx. comendo (N=5/48/96)	Total N=96
V0	nenhuma (sequencial)	não	1 / 1 / 1	38.9 s
V1	1 mutex por garfo, esq→dir	sim (30/30)	0	não termina
V2	mutex global + garfos	não	1 / 1 / 1	19.5 s
V3	garfos ordenados (menor índice 1º)	não	2 / 23 / 44	0.53 s

Análise comparativa (resumo)

V3 é a melhor: com N=96 leva 0.53 s contra 19.5 s da V2 (37× mais lenta) e 38.9 s da V0 (73×), com até 44 filósofos comendo em paralelo. V2 é correta, mas só ganha ~2.0× sobre a V0 (sobrepõe apenas o pensar) e a espera média cresce com N (29.769 → 927.872 ms). V1 travou em 30 de 30 execuções.

Resultados experimentais

Versão	N	Exec. (ok/dead)	Total médio (ms)	Desvio (ms)	Mín-Máx (ms)	Espera média (ms)	Maior espera (ms)	Máx. comendo	Ciclos/ filósofo
V0	5	5/0	2025.3	0.4	2024.5–2025.5	0.000	0.003	1	20–20
V0	48	5/0	19435.0	0.2	19434.7–19435.2	0.000	0.000	1	20–20
V0	96	5/0	38870.2	0.5	38869.8–38871.0	0.000	0.000	1	20–20
V1	5	0/10	deadlock	-	-	-	-	-	parcial: 0
V1	48	0/10	deadlock	-	-	-	-	-	parcial: 0
V1	96	0/10	deadlock	-	-	-	-	-	parcial: 0
V2	5	5/0	1022.7	0.0	1022.7–1022.8	29.769	50.613	1	20–20
V2	48	5/0	9730.2	0.8	9729.6–9731.5	454.154	486.072	1	20–20
V2	96	5/0	19450.8	1.2	19448.8–19451.7	927.872	972.037	1	20–20
V3	5	5/0	534.7	8.4	526.6–546.8	5.003	20.370	2	20–20
V3	48	5/0	511.6	30.1	467.0–537.9	2.443	40.581	23	20–20
V3	96	5/0	532.8	15.4	508.4–549.4	3.335	50.828	44	20–20

Tabela 1 — Médias sobre as execuções concluídas (c8g.16xlarge, 20 ciclos, pensar=comer=10 ms). Dados crus: c8g.16xlarge/results.csv. Na V1 o “tempo” medido seria só o do watchdog (5,1 s) e não é comparável.

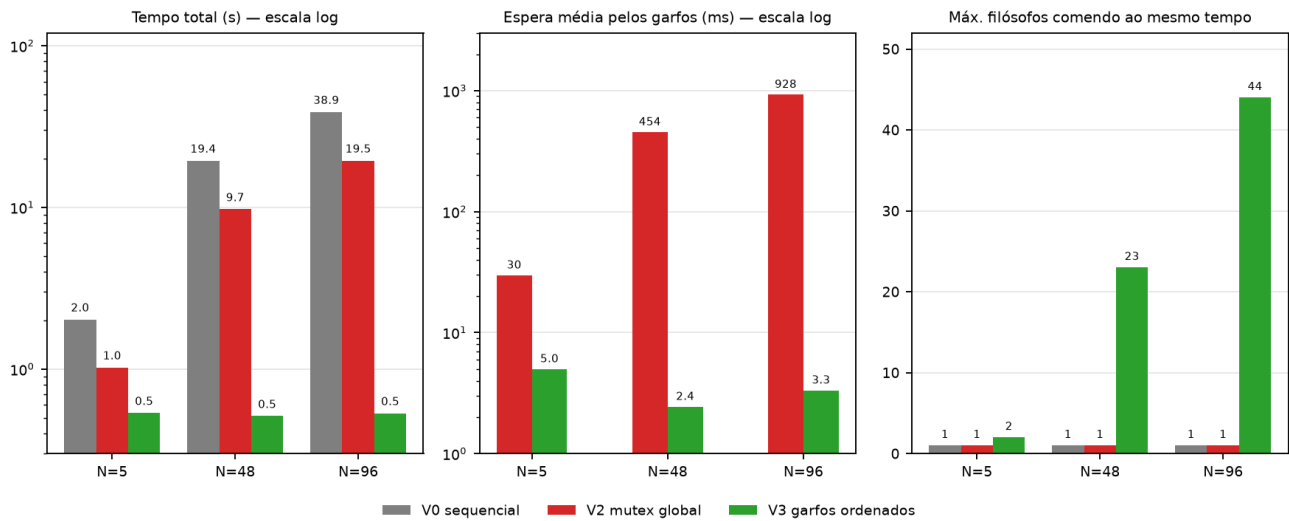


Figura 1 — Comparação por versão e N (barras: média das 5 execuções).

Captura 1 — V1 em deadlock (saída real do terminal, ./filosofos -v 1 -n 5)

```
$ ./filosofos -v 1 -n 5
V1 n=5 status=DEADLOCK
tempo total:      5100,51 ms
espera média:     0.000 ms
maior espera:     0.000 ms
máx. comando ao mesmo tempo: 0
ciclos por filósofo: 0 0 0 0 0

== Evidência de deadlock ==
filósofo 0: tem garfo 0, ESPERA garfo 1 (em posse do filósofo 1) | ciclos=0
filósofo 1: tem garfo 1, ESPERA garfo 2 (em posse do filósofo 2) | ciclos=0
filósofo 2: tem garfo 2, ESPERA garfo 3 (em posse do filósofo 3) | ciclos=0
filósofo 3: tem garfo 3, ESPERA garfo 4 (em posse do filósofo 4) | ciclos=0
filósofo 4: tem garfo 4, ESPERA garfo 0 (em posse do filósofo 0) | ciclos=0
```

Captura 2 — linhas reais do results.csv (versao, n, status, total_ms, espera_media_ms, espera_max_ms, max_comando, ciclos)

[illegible]

Conclusão e análise dos resultados

1. Como funciona cada solução. V0 executa os filósofos em sequência (baseline). V1 pega esquerdo e depois direito e trava. V2 envolve pegar-comer-soltar num mutex global. V3 impõe ordem global de aquisição dos garfos (menor índice primeiro).

2. Mecanismos. Exclusão mútua por `sync.Mutex` (um por garfo, mais um global na V2), `WaitGroup` para juntar goroutines, `atomic` para contadores e um `watchdog` para detectar deadlock. Nenhum canal foi necessário.

3. Implementação em Go. Mutex do runtime (sem preempção, não reentrante) sobre semáforos; goroutines escalonadas em até 64 threads do SO (`GOMAXPROCS = 64`); `time.Sleep` simula pensar/comer sem consumir CPU, por isso o desempenho é dominado pela *estrutura de sincronização*, não pelo número de cores.

4. Deadlock na V1. As quatro condições de Coffman ocorrem ao mesmo tempo (página 1). A evidência coletada é o estado de cada filósofo no travamento: todos com 1 garfo (o esquerdo) esperando o direito, formando o ciclo $0 \rightarrow 1 \rightarrow \dots \rightarrow N-1 \rightarrow 0$, com 0 ciclos concluídos. Travou em **30 de 30** execuções (N=5, 48 e 96). O *gap* de 1 ms entre os dois garfos alinha os filósofos e torna isso quase certo; em testes locais preliminares (outra máquina, sem controle) algumas execuções chegaram a terminar, ou seja, o deadlock depende de timing. Sem o gap não medimos a probabilidade.

5. Como V2 e V3 evitam. V2 impede *segura-e-espera* e a espera circular (um só filósofo por vez disputa garfos). V3 impede a espera circular pela ordenação: como o último filósofo pega o garfo 0 antes do N-1, não existe ciclo de dependências. Ambas concluíram 100% das execuções (V2: 15/15, V3: 15/15).

6. Starvation. Em teoria a V3 usa só ordenação e não garante justiça, e a V2 depende da fila do mutex global. Na prática o modo de inanição do `sync.Mutex` (1 ms) faz a espera ser limitada. Medimos isso: todos os filósofos fizeram exatamente 20 ciclos em todas as execuções, e a maior espera foi 972.037 ms na V2 ($\approx N \times 10$ ms = 960 ms: cada um espera a fila inteira comer) e 50.828 ms na V3 (~ 5 tempos de comer). Não observamos starvation; a V1 não se aplica, pois trava antes.

7. Maior tempo total. Entre as que terminam, a **V0** (N=96: 38.9 s; N=48: 19.4 s; N=5: 2.03 s), que coincide com o previsto $N \times 20 \times 20$ ms. A V1 não termina (tempo infinito).

8. Maior tempo de espera. A **V2**: média de 29.769 ms (N=5), 454.154 ms (N=48) e 927.872 ms (N=96), crescendo linearmente com N, contra 5.003–3.335 ms na V3 e ~ 0 na V0 (não há disputa). Na V1 a espera é infinita.

9. Filósofos comendo simultaneamente. V0: 1. V2: 1 (mesmo com garfos livres). V3: 2 (N=5), 23 (N=48) e 44 (N=96), próximo do teto teórico $N/2 = 2, 24$ e 48 (arredondando para baixo). V1: 0 (ninguém chega a comer).

10. Impacto da sincronização excessiva (V2). A V2 só é $\sim 2.0 \times$ melhor que a V0: o mutex global já permite sobrepor o *pensar* de uns com o comer de outro, mas serializa o comer. Frente à V3 ela é $1.9 \times$ mais lenta com N=5, $19 \times$ com N=48 e $37 \times$ com N=96 — o custo cresce com N porque a V3 mantém o tempo quase constante e a V2 cresce linearmente.

11. Mais sincronização = melhor desempenho? Não. A V2 é a que mais sincroniza (mutex global *além* dos garfos) e foi a pior das corretas, com maior espera e concorrência 1. A V3 usa a *mesma* quantidade de locks de garfo, mas sem o ponto de serialização, e teve o menor tempo total e a menor espera. Importa *onde* se sincroniza (escopo da seção crítica), não quanto. Também não é mais sincronização o que evita deadlock, e sim a estrutura (ordem) dela.

12. Grau de concorrência $\times$ tempo total. O tempo é aproximadamente $T \approx (\text{trabalho a serializar}) \div (\text{concorrência efetiva})$: V0 (concorrência 1): $T = N \times 20 \times 20$ ms; V2 (o comer é serial, o pensar sobrepõe): $T \approx N \times 20 \times 10$ ms (medido 19.45 s vs 19,2 s previstos com N=96); V3 (concorrência $\approx N/2$): $T \approx 20 \times (\text{pensar} + \text{comer}) = 400$ ms, medido ≈ 533 ms (+30%, por disputa de garfos vizinhos e jitter de timers). Speedup da V3 sobre a V0: $3.8 \times$ (N=5), $38 \times$ (N=48), $73 \times$ (N=96), acompanhando o aumento da concorrência observada ($2 \rightarrow 23 \rightarrow 44$).

Comparação: c8g.medium (1 vCPU) $\times$ c8g.16xlarge (64 vCPUs)

Versão	N	Total (ms) medium	Total (ms) 16xlarge	Espera média (ms) medium / 16xl	Máx. comendo medium / 16xl
V0	5	2027.7	2025.3	0.000 / 0.000	1 / 1
V0	48	19419.3	19435.0	0.000 / 0.000	1 / 1
V0	96	38838.6	38870.2	0.000 / 0.000	1 / 1
V2	5	1021.5	1022.7	29.834 / 29.769	1 / 1
V2	48	9721.7	9730.2	453.961 / 454.154	1 / 1
V2	96	19431.4	19450.8	927.301 / 927.872	1 / 1
V3	5	556.4	534.7	4.451 / 5.003	2 / 2
V3	48	426.2	511.6	0.750 / 2.443	24 / 23
V3	96	437.8	532.8	0.705 / 3.335	47 / 44

A medium (1 vCPU, `GOMAXPROCS=1`) repetiu o mesmo experimento (5 execuções por caso; V1 com 10). **V0 e V2 ficaram idênticos** (razão 0,999–1,001): são limitados por *sleep* e por serialização, não por CPU. **A V1 travou nas**

30 execuções também na medium: o deadlock é lógico e não depende de núcleos. **A V3 foi ~17–18% mais rápida na medium** com N=48 e 96 (e ~4% mais lenta com N=5) (437.8 ms contra 532.8 ms com N=96), com espera média menor (0.705 contra 3.335 ms), desvio-padrão quase zero (0.1 ms contra 15.4 ms) e concorrência máxima de 47 filósofos (teto 48). Hipótese, não verificada: com um só P, os timers acordam os filósofos em lotes alinhados e há menos migração/jitter entre threads; na 16xlarge, o escalonamento em 64 threads adiciona atrasos (mais variação). Conclusão: para esta carga, mais cores **não** melhoram o desempenho; a estrutura de sincronização é o fator dominante (V2 → V3 = 19–37×), e um núcleo basta.

Ressalvas

- O enunciado usa N=5; N=48 e 96 foram acrescentados para evidenciar a escala. Em N=5 o ganho da V3 sobre a V2 é modesto (1,9×) e a V3 é limitada a 2 comensais.
- Pensar/comer são simulados com *sleep*; com trabalho real de CPU, o número de cores passaria a limitar a V3. Desvios-padrão baixos (≤ 31 ms) indicam medidas estáveis; 5 execuções por caso.
- Medidas de espera incluem a espera pelo mutex global (V2) e pelos dois garfos (V1/V3). A V1 é analisada à parte: as 30 execuções terminaram em deadlock, com evidência em `deadlock_n5/48/96.txt`.
- As “capturas” são a saída textual real do terminal; código completo no repositório indicado na página 1.